

Input Validation & Errors

Right now we can create a new job but if we try and submit an empty form it will break because the title field can not be null. We need to add some validation to the form. We will use Laravel's built-in validation rules.

We have access to the request object in the controller. We can use the `validate` method to validate the request. The `validate` method will throw an exception if the validation fails. We can use the `withErrors` method to add the errors to the session. We can then use the `old` method to display the errors in the view.

Let's add some validation to the `store` method in the `JobListingController`. Add the following code to the `store` method:

```
public function store(Request $request): RedirectResponse
{
    // Validate the incoming request data
    $validatedData = $request->validate([
        'title' => 'required|string|max:255',
        'description' => 'required|string',
    ]);

    // Create a new job listing with the validated data
    Job::create([
        'title' => $validatedData['title'],
        'description' => $validatedData['description'],
    ]);

    return redirect()->route('jobs.index');
}
```

In this code, we are using the `validate` method to validate the incoming request data. We are passing an array of validation rules to the `validate` method. The `validate` method will return the validated data if the validation passes. If the validation fails, it will throw an exception.

We are using the `required` rule to make sure the title and description fields are not empty. We are using the `string` rule to make sure the title is a string. We are using the `max` rule to make sure the title is no longer than 255 characters. You can find all of the validation rules [here](#).

If the validation passes, we are creating a new job listing with the validated data. If the validation fails, the user will be redirected back to the form with the errors.

Displaying Errors With `@error`

We can display the errors with the `@error` directive. Add the following code to the `/resources/views/jobs/create.blade.php` file:

```
<form action="/jobs" method="POST">
    @csrf
```

```
<input type="text" name="title" placeholder="Title">
<!-- Error Message for Title -->
@error('title')
<div class="text-red-500 mt-2 text-sm">{{ $message }}</div>
@enderror
<input type="text" name="description" placeholder="Description">
<!-- Error Message for Description -->
@error('description')
<div class="text-red-500 mt-2 text-sm">{{ $message }}</div>
@enderror
<button type="submit">Submit</button>
</form>
```

Again, this is very ugly. We will fix the styling later after we add the rest of the fields.

Now if you submit the empty form you will see the errors. We still have an issue though. Try and just submit the title and leave the description blank. You see the error but the value that you typed goes away. We need to add the `old` method to the form values.

old Method

The `old` method is used to repopulate the form fields with the old input. This is useful when the form fails validation and you want to repopulate the form with the old input. Add the following code to the `/resources/views/jobs/create.blade.php` file:

```
<form action="/jobs" method="POST">
  @csrf
  <input type="text" name="title" placeholder="Title" value="{{ old('title') }}">
  <!-- Error Message for Title -->
  @error('title')
  <div class="text-red-500 mt-2 text-sm">{{ $message }}</div>
  @enderror
  <input type="text" name="description" placeholder="Description" value="{{ old('description') }}">
  <!-- Error Message for Description -->
  @error('description')
  <div class="text-red-500 mt-2 text-sm">{{ $message }}</div>
  @enderror
  <button type="submit">Submit</button>
</form>
```

Now only fill in the title and click submit. You will see the title is repopulated with the old input.